

Deployed Agent API with an Automated Workflow

Kayfa Academy · Ain Shams University — give the agent an address (S5), then take your hands off the trigger (S6).

COVERS	Session 5 — Production Backend with FastAPI Session 6 — Advanced Workflow Automation with n8n	WEIGHT	100 points (+10 bonus)
WORK MODE	Individual (or pairs, if approved by the instructor)	DUE	_____ before the capstone submission

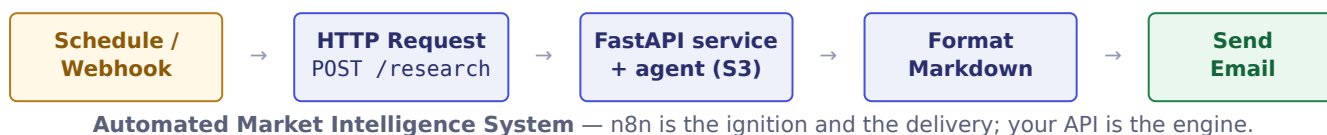
1. The mission

In Task 2 your agent gained a face. It still only ran when *you* ran it. Task 3 removes you from the loop entirely: the agent becomes a **service with an address** that anything on the internet can call, and then a **schedule or webhook** calls it on your behalf and delivers the result to a human inbox.

Deliver one pipeline, not two exercises. The FastAPI service and the n8n workflow are graded as a single system: n8n must call *your* API, and the email must contain *your* agent's output.

The one moment that must work. A run that **you did not start**. The trigger fires, the API is called, the agent runs, a formatted report lands in an inbox — and the n8n execution log proves nobody clicked anything. That unattended execution is the deliverable; everything else supports it.

System you are delivering



2. Part A — The FastAPI service (Session 5)

Wrap the Task 2 agent in a production-grade API. The agent code is imported, not rewritten — if you kept the seam clean in Task 2, this is a thin layer on top.

A.1 — Required endpoints

ENDPOINT	METHOD	BEHAVIOUR
<code>/research</code>	POST	Validated request → async agent run → structured JSON response. The primary endpoint; this is what n8n calls.
<code>/research/stream</code>	POST	Streams the agent's progress as it runs, via <code>StreamingResponse</code> over <code>agent.astream(...)</code> .
<code>/research/async</code>	POST	Accepts the job, returns <code>202 Accepted</code> immediately, runs the work in <code>BackgroundTasks</code> .

/health	GET	Liveness check returning status and version. No auth, no agent call, fast.
---------	-----	--

A.2 — The contract

Define both directions with Pydantic. Keep these field names — the n8n nodes and the rubric depend on them.

```
class ResearchRequest(BaseModel):
    topic: str # required - the research objective
    max_retries: int = 2 # optional override of the S3 retry cap

class ResearchResponse(BaseModel):
    topic: str
    report: str # the final Markdown / structured report
    quality_score: float
    retry_count: int
    tokens_used: int
    duration_seconds: float
```

Pydantic from Session 1 is back. A malformed request must be rejected at the gate with a clean 422 — before it ever reaches your agent and spends money.

A.3 — Production concerns (all four required)

- **Rate limiting** — `slowapi` or equivalent middleware, capped per client per minute. State your chosen limit in the README and be ready to show a 429.
- **Background tasks** — the long job returns immediately rather than holding the caller for 60 seconds.
- **Structured logging** — JSON logs carrying a request ID, endpoint, duration and outcome for every call.
- **Validation** — Pydantic models on the boundary, in both directions.

A.4 — Tests

Assertions using FastAPI's `TestClient`, runnable in the notebook with no server:

- A valid payload returns 200 and a body matching `ResearchResponse`.
- A malformed payload (missing `topic`, or wrong type) returns 422.
- `/health` returns 200.
- Exceeding the rate limit returns 429.

3. Part B — The n8n workflow (Session 6)

#	NODE	REQUIREMENT
1	Trigger	Schedule (e.g. daily at 08:00) or Webhook. The trigger must be active , not fired by pressing "Execute workflow" by hand.
2	HTTP Request	<code>POST</code> to your <code>/research</code> endpoint with a JSON body containing the topic. Timeout raised to cover a full agent run.
3	Parse / summarize	Read the JSON response and pass the report through a summarization or transformation step.
4	Format Markdown	Build a readable report — title, date, headings, bullet points, and the quality score / retry count from the response.
5	Send Email	Gmail or SMTP node delivering to a real inbox. Credentials live in n8n's credential store — never in the node body, never in the exported JSON.
6	Error path	Handle a failed API call — the workflow must not die silently when the agent is down. A notification or an error branch is enough.

n8n cannot reach localhost. If you run n8n in the cloud and your API on your machine, the HTTP Request node will fail. Either expose your API with a tunnel (`ngrok` / `localtunnel`), deploy it, or run n8n locally alongside it. Solve this *before* the demo, not during it.

4. Suggested project structure

```

task3/
├── agent/                # imported unchanged from Task 2
├── api/
│   ├── main.py          # FastAPI app, routes
│   ├── schemas.py       # ResearchRequest / ResearchResponse
│   ├── limits.py        # rate limiting config
│   └── logging_conf.py  # structured JSON logging
├── tests/test_api.py    # TestClient assertions
├── n8n/workflow.json    # exported workflow - credentials stripped
├── requirements.txt
├── .env.example
└── README.md           # run steps, tunnel setup, screenshots

```

5. Deliverables

1. **Source code** — repository link (or `.zip`), structured as above.
2. **Exported n8n workflow** — importable `workflow.json`, with credentials removed.
3. **Screenshots** — the `/docs` Swagger UI, the n8n canvas, a successful execution log, and the delivered email.
4. **Test output** — passing `TestClient` assertions, including the `422` and `429` cases.
5. **Sample run** — evidence of one **unattended** execution: the n8n execution history entry showing the trigger type and timestamp, next to the email it produced.
6. **README** — how to run the API, how to import the workflow, and how you connected the two.

6. Grading rubric

CRITERION	WHAT EARNS FULL MARKS	POINTS
Endpoints & validation	All four endpoints implemented; request and response models enforced; clean <code>422</code> on bad input; responses match the published contract.	25
Async & streaming	Handlers are genuinely async (no blocking calls in the event loop); the streaming endpoint emits progress incrementally, not one blob at the end.	10
Production concerns	Rate limiting enforced and demonstrable; background task returns <code>202</code> immediately; structured logs with request IDs; working health check.	15
Tests & docs	TestClient assertions cover success, validation failure, health and rate limit; <code>/docs</code> renders every endpoint correctly.	10
Workflow trigger & API call	Active schedule or webhook trigger; HTTP Request node successfully calls your API and receives valid JSON; an unattended run is evidenced.	15
Formatting & delivery	Report is readable Markdown, not a raw JSON dump; email arrives reliably; an error path exists for a failed call.	10
Code quality & secrets	API layer separate from agent layer; no keys or credentials in the repo or the exported workflow; pinned requirements.	10
Documentation & sample run	README lets someone else stand the system up; screenshots and run evidence are complete and legible.	5
Total		100

Bonus (up to +10)

- Reprint your Task 2 Streamlit dashboard at the API over HTTP instead of in-process — same UI, new transport (+4).
- Deploy the API publicly (Docker, Render, Railway, Fly.io) and have n8n call the live URL (+3).
- Both trigger types working: a scheduled digest **and** a webhook that runs research on demand (+3).

7. Before you submit

- ☐ Fresh clone, fresh virtualenv, `uvicorn main:app` — the service starts with no manual patching.
- ☐ `/docs` lists all four endpoints and every one can be called from the browser.
- ☐ An invalid payload returns `422`, not a 500 traceback.
- ☐ Hammering the endpoint produces a `429` — the limit is real, not just imported.
- ☐ Logs contain a request ID and a duration for each call.
- ☐ The exported `workflow.json` contains no API keys, tokens or passwords.
- ☐ One execution in the n8n history was started by the trigger, not by you.
- ☐ The delivered email is formatted Markdown a human would actually read.

8. Common pitfalls

SYMPTOM	CAUSE / FIX
n8n reports connection refused	The API is on <code>localhost</code> and n8n is not. Tunnel it, deploy it, or run n8n locally.
HTTP node times out mid-run	Agent takes longer than the node's default timeout. Raise it, or call <code>/research/async</code> and collect the result separately.
Endpoint is <code>async</code> but still blocks	A synchronous LLM/agent call inside an async handler. Use the async client, or offload with <code>run_in_threadpool</code> .
Email body is raw JSON	The formatting step was skipped. Build Markdown from the fields — readability is graded.
Workflow works once, never again	Executed manually instead of activated. Toggle the workflow active and let the trigger fire.
Credentials leaked in the export	Secrets typed into node fields instead of the credential store. Move them and re-export.

Where this goes next. With Task 3 delivered you hold the whole mandatory stack — LangGraph, LlamaIndex, Streamlit, FastAPI and n8n — wired end to end. The capstone is not new machinery; it is this pipeline pointed at a real problem you care about. Build it well now and the capstone is mostly a change of topic.